

Cluster Resource Manager.

The second resource `nginx` is the name of the start-up script to start/stop the resource `nginx` and by default, Heartbeat looks for it in either `/etc/init.d` or `/etc/ha.d/resource.d`. The lines in `haresources` are translated as follows:

- On Heartbeat startup, acquire IP address 10.8.0.1 and then start `nginx` on node `rproxy1`
- When Heartbeat is stopped, stop `nginx` and then release IP 10.8.0.1 on node `rproxy1`

The `authkeys` file

The `authkeys` file is very important in maintaining the security of the cluster, as it authenticates the cluster nodes. This file should be owned by the root with permissions set to 600 (that is, readable and writeable by only the root), otherwise Heartbeat will refuse to start. Also, all the nodes of the cluster should have an identical `authkeys` file. Heartbeat supports three authentication methods: `crc`, `md5` and `sha1`. You probably don't use a serial or crossover connection for Heartbeat—if you do, `crc` is a good choice. For set-ups where the Heartbeat travels over the network, `sha1` is a good choice. We will use `sha1` in our set-up.

The following is the `authkeys` file in our set-up:

```
# cat /etc/ha.d/authkeys
auth 1
1 sha1 ThisIsMySecretKeyAndICanChooseAnyStringHere
```

Typically, the number of the key used is 1, but you can use any number ranging from 1-15. Just make sure that whatever number you use with the `auth` line, is present in one of the keys listed in the following lines. In order to generate a complete random number as a secret key for `sha1`, use the following command as suggested in the Linux-HA website and other places on the Web, and replace the string “ThisIsMySecretKeyAndICanChooseAnyStringHere” with the output of the following command:

```
# dd if=/dev/urandom count=4 2>/dev/null | openssl dgst -sha1
```

Controlling the start-up of cluster resources

Since our proxy server functionality is dependent on the IP address, we need to make sure that the `nginx` server does not get started automatically by the system start-up scripts during system boot. On Debian and its derivatives, this can be done as follows:

```
# invoke-rc.d nginx stop
# update-rc.d -f nginx remove && update-rc.d nginx stop 45
```

Note how the `update-rc.d` script is used above. If we would have just removed the `nginx` from start-up as done by the `remove` command, an update to `nginx` through `apt-get` would have triggered the creation of the missing symbolic links in the `rc.d` directories. This is not what we wanted and

IPaddr versus IPaddr2 Cluster Resource Manager

Heartbeat uses either the `IPaddr` or `IPaddr2` resource manager script to configure IPv4 service addresses. By default, the `IPaddr` script is used. The cluster resource manager scripts are located in the `/etc/ha.d/resource.d` directory. The basic syntax for both the scripts are:

```
IPaddr ip-address[/netmask[/interface[/broadcast]]
IPaddr2 ip-address[/netmask[/interface[/broadcast]]
```

The difference between the two is that the `IPaddr` script uses the ancient method of IP aliasing, whereas `IPaddr2` uses the new IPv2 method of setting secondary IP address. There is a limit of 100 aliases on the `IPaddr` script, while there is no such limit on `IPaddr2` script. In case you use `IPaddr2` (which is preferable, and the default cluster resource manager script in Heartbeat now), you will not be able to see the acquired service address using the `ifconfig` command. This can be done by using the `ip addr show` command. For details on the `ip` command, refer to the “Linux Advanced Routing and Traffic Control” how-to link in the References section.

will bring our cluster down. For post installation/update scripts not to create or update symbolic links, and in order to leave `nginx` in the default disabled state, we have created a stop symlink in run-level 4 and 5 (`/etc/rc4.d/K?nginx` and `/etc/rc5.d/K?nginx`) file. This is because `update-rc.d` has been designed in a way to ignore creating or updating any symlinks, if something like `/etc/rc.d/SK?name` (where ‘name’ is `nginx`, in our case) already exists. This makes sure that a package update never changes any existing configuration. For details, read the man page of `update-rc.d`.

On RHEL and derivatives:

```
# /sbin/service nginx stop
# /sbin/chkconfig nginx remove
```

Starting the cluster

Now that we are all set to start the cluster, execute the following commands as per your distribution:

```
# invoke-rc.d heartbeat start ## on Debian
or:
# /sbin/service heartbeat start ## on RHEL
```

Our cluster will be up and running in a short time. You can check the availability of the service address and running `nginx` instance. In order to test the cluster failover and fallback, you can start playing with various options like unplugging the network cable from the back of the primary node, switching it off physically, etc.

I will leave testing in your capable hands.

Further tuning

I would strongly recommend that you set up the cluster to use the `ipfail` plug-in to ensure proper failover, if a network